

House Price Prediction: End-to-End Machine Learning Regression Notes

Formal English reference notes based on the workflow discussed for the house-price regression project.

1. Problem Definition

The objective is to predict house prices from selected property features. This is a supervised learning regression problem because the target, price, is a continuous numerical value.

2. Feature Selection

Feature selection determines which variables are supplied to the model. In this project, identifier, redundant, and selected unnecessary columns were removed. X contains input features and y contains the price target.

3. Exploratory Data Analysis

EDA is performed before modeling to understand distributions, relationships, correlations, outliers, missing values, and target behavior. It supports informed preprocessing and modeling decisions.

4. Data Visualization

Histograms, box plots, scatter plots, correlation heatmaps, and target-distribution plots help visually understand the dataset. Visualization supports analysis but does not replace quantitative model evaluation.

5. Train-Test Split

The data is divided into training and testing subsets. The training set is used to learn the model, while the test set is reserved for evaluating performance on unseen observations.

6. Feature Scaling

StandardScaler standardizes numerical features using statistics learned from the training data. The scaler is fitted only on training data and then applied to both training and test data.

7. Regression Models

The project compares Linear Regression, Ridge, Lasso, ElasticNet, Polynomial Regression, KNN, SVR, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, AdaBoost, XGBoost, LightGBM, and CatBoost.

8. Linear Regression

Linear Regression predicts the target as a linear combination of input features. It is simple, interpretable, and useful as a baseline.

9. Ridge Regression

Ridge adds L2 regularization to linear regression. It discourages excessively large coefficients and can improve stability when features are correlated.

10. Lasso Regression

Lasso uses L1 regularization. It can shrink some coefficients to zero, providing both regularization and implicit feature selection.

11. ElasticNet Regression

ElasticNet combines L1 and L2 regularization and can be useful when features are correlated while still encouraging a simpler model.

12. Polynomial Regression

Polynomial Regression creates polynomial features so a linear estimator can represent nonlinear relationships. Higher complexity can increase overfitting.

13. KNN Regression

KNN predicts a value using nearby training observations. Because it relies on distances, feature scaling is important. It can become computationally expensive with larger datasets.

14. Support Vector Regression

SVR applies support-vector methods to continuous targets. Kernel functions can represent nonlinear relationships. SVR is sensitive to C, gamma, and epsilon and can be slow on larger datasets.

15. Decision Tree Regression

A Decision Tree recursively partitions the feature space and predicts a value in each region. It handles nonlinear relationships without scaling but can overfit if unrestricted.

16. Random Forest Regression

Random Forest is a bagging ensemble of decision trees. It combines many randomized trees to reduce variance and produce more stable predictions.

17. Extra Trees Regression

Extra Trees introduces additional randomness when constructing trees. This can reduce variance and provide strong performance on nonlinear tabular data.

18. Gradient Boosting Regression

Gradient Boosting builds trees sequentially, with each new tree improving errors made by the existing ensemble. It can model complex nonlinear relationships.

19. AdaBoost Regression

AdaBoost sequentially emphasizes difficult observations and combines weak learners into a stronger predictor.

20. XGBoost

XGBoost is an optimized gradient boosting implementation with regularization and engineering optimizations designed for strong predictive performance and efficiency.

21. LightGBM

LightGBM is an efficient gradient boosting framework designed particularly for structured and larger datasets. Its histogram-based learning provides fast training.

22. CatBoost

CatBoost is a gradient boosting algorithm with specialized handling for categorical features. In the current project, the selected features are primarily numerical, so CatBoost is mainly used as a strong boosting model.

23. Ensemble Learning

Ensemble learning combines multiple models so that different strengths and errors can contribute to a final prediction. An ensemble is not automatically better and must be evaluated against individual models.

24. Voting Regressor

Voting Regressor combines several regression estimators, commonly by averaging their predictions. Weighted voting can give stronger models greater influence.

25. Stacking Regressor

Stacking uses base-model predictions as inputs to a meta-model. The meta-model learns how to combine the base predictions instead of simply averaging them.

26. Regression Metrics

MAE measures average absolute error and lower is better. RMSE is more sensitive to large errors and lower is better. R^2 measures explained variance and higher is better. R^2 should not be called classification accuracy.

27. Model Comparison

Each candidate model is evaluated using the same evaluation framework. Results are stored with Model, R^2 Score, MAE, and RMSE and then ranked. The final decision should consider all metrics.

28. ML Pipeline

A pipeline combines preprocessing and model training into one reproducible object. Scaled models can contain StandardScaler followed by the estimator, while tree-based models generally do not require scaling.

29. Automated Model Comparison

A dictionary of models can be iterated through so every estimator is trained, evaluated, and stored using consistent code. This reduces repetition and makes experiments easier to maintain.

30. Hyperparameter Tuning

Hyperparameters are model settings such as tree depth, learning rate, iterations, and regularization. Tuning searches for settings that improve validation performance.

31. GridSearchCV

GridSearchCV evaluates every specified hyperparameter combination using cross-validation. It can be slow because the number of model fits grows rapidly with the grid and number of folds.

32. RandomizedSearchCV

RandomizedSearchCV evaluates a selected number of parameter combinations instead of every combination. It often provides a faster first tuning strategy.

33. CatBoost Tuning

The project used iterations, depth, learning rate, and `l2_leaf_reg` in a small randomized search. If tuning produces almost the same result as the baseline, further tuning may not be worthwhile.

34. CatBoost Visualization

Useful CatBoost visualizations include Actual-vs-Predicted, residual plots, residual distributions, feature importance, and training/validation learning curves.

35. Actual vs Predicted

Predicted values are plotted against actual values. Points close to the diagonal reference line indicate more accurate predictions.

36. Residual Analysis

A residual is the difference between actual and predicted values. Residual plots can reveal systematic errors, nonlinearity, changing variance, and other model deficiencies.

37. Feature Importance

Feature importance ranks variables according to their contribution to a tree-based model. It is model-specific evidence and should not be interpreted as proof of causation.

38. Final Model Selection

The final model should be chosen after comparing candidates, validating promising models, and tuning where useful. A strong candidate should have high R^2 and low MAE and RMSE while generalizing well.

39. Saving the Final Model

The complete trained pipeline can be serialized with joblib and saved as a .pkl file. Saving the complete pipeline preserves preprocessing and the trained estimator together.

40. Deployment

The saved pipeline can be loaded by applications such as Streamlit or APIs such as FastAPI. New input data can be passed through the same preprocessing and prediction workflow.

41. Complete Workflow

The complete project follows: data preparation → feature selection → EDA → visualization → train-test split → preprocessing → model comparison → evaluation → promising-model selection → cross-validation and tuning → ensemble comparison where useful → final evaluation → save pipeline → deployment.

Regression Metric Rules

MAE: Lower is better. It represents the average absolute prediction error.

RMSE: Lower is better. It penalizes large prediction errors more strongly than MAE.

R² Score: Higher is better. It measures the proportion of target variance explained by the model.

Important: For regression, R² should be reported as R² Score rather than as classification accuracy.

Recommended final workflow: Understand the data and algorithms first, automate experiments with pipelines, compare all baseline models, tune only promising candidates, evaluate the final model carefully, save the complete pipeline, and then deploy it.